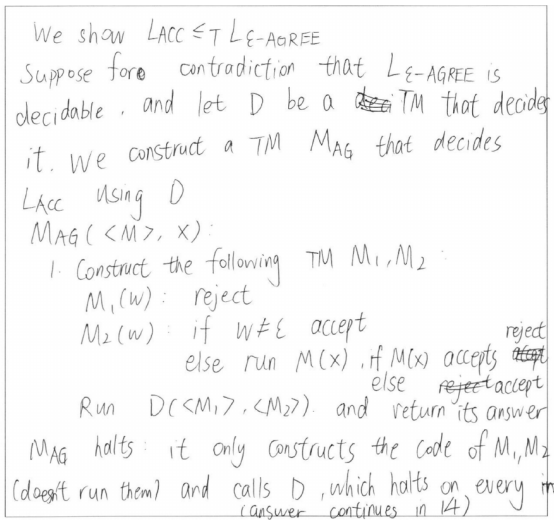
Q0 Before you start; before you submit

I read Handout 6 on NP-hardness proofs before starting. My reductions follow its steps: describe the reduction, show it runs in polynomial time, and prove both directions of correctness. I did not collaborate with anyone on this homework.

Q1 Self assessment

I chose Question 17 of the midterm, the Turing reduction $L_{\text{ACC}} \leq_T L_{\varepsilon\text{-AGREE}}$. The part with the most room for improvement is my proof of correctness. Here is what I submitted (the answer box on page 13 and its continuation on page 14):



We show $L_{ACC} \leq_T L_{\varepsilon\text{-AGREE}}$
Suppose for contradiction that $L_{\varepsilon\text{-AGREE}}$ is decidable, and let D be a TM that decides it. We construct a TM M_{AG} that decides L_{ACC} using D
$M_{AG}(\langle M \rangle, x)$:
1. Construct the following TM M_1, M_2:
$M_1(w)$: reject
$M_2(w)$: if $w \neq \varepsilon$ accept
else run $M(x)$, if $M(x)$ accepts reject else accept
Run $D(\langle M_1 \rangle, \langle M_2 \rangle)$ and return its answer
M_{AG} halts: it only constructs the code of M_1, M_2 (doesn't run them) and calls D, which halts on every input
(answer continues in 14)

input.
Correctness:
$(\langle M \rangle, x) \in L_{ACC} \Rightarrow M(x)$ accepts $\Rightarrow M_2(\varepsilon)$ rejects
$\Rightarrow M_1(\varepsilon) = M_2(\varepsilon)$ and $\forall x \neq \varepsilon \; M_1(x) \neq M_2(x) \Rightarrow D$ accepts $\Rightarrow$ M_{AG} accepts
$(\langle M \rangle, x) \notin L_{ACC} \Rightarrow M(x)$ rejects $\Rightarrow M_2(\varepsilon)$ accepts
$\Rightarrow M_2(\varepsilon) \neq M_1(\varepsilon) \Rightarrow D$ rejects $\Rightarrow M_{AG}$ rejects
Conclusion: M_{AG} decides L_{AC}. But L_{AC} is undecidable
Hence $L_{\varepsilon\text{-AGREE}}$ is undecidable

What was deficient. My construction is correct, but the second half of my correctness proof is wrong. For $(\langle M \rangle, x) \notin L_{\text{ACC}}$ I wrote "$\Longrightarrow M(x)$ rejects $\Longrightarrow M_2(\varepsilon)$ accepts." Not being in L_{ACC} only means that $M(x)$ does *not accept*: it may reject, or it may loop. I only handled the case where it rejects, so the case where $M(x)$ loops is missing. The posted solution writes "rejects/loops" at this step, and I lost the points for this direction. There are also two smaller problems. In the first chain I wrote "$\forall x \neq \varepsilon$, $M_1(x) \neq M_2(x)$" without explaining why. I also used the letter x there for the input of M_1, M_2, but x is already the input of M.

How to fix it. I should first say what is true in both directions: M_1 rejects every input, and for every $w \neq \varepsilon$, $M_2(w)$ accepts without running M. So $M_1(w) \neq M_2(w)$ for all $w \neq \varepsilon$. This means $(\langle M_1 \rangle, \langle M_2 \rangle) \in L_{\varepsilon\text{-AGREE}}$ if and only if $M_2(\varepsilon)$ rejects, like $M_1(\varepsilon)$ does. Then in the second direction I need two cases. If $M(x)$ rejects, $M_2(\varepsilon)$ accepts. If $M(x)$ loops, $M_2(\varepsilon)$ never finishes simulating $M(x)$, so it loops. In both cases $M_2(\varepsilon)$ does not reject, so $M_1(\varepsilon) \neq M_2(\varepsilon)$, the pair is not in $L_{\varepsilon\text{-AGREE}}$, D rejects, and M_{AG} rejects. So the second chain should start with "$M(x)$ rejects or loops $\Longrightarrow M_2(\varepsilon)$ accepts or loops," and the rest stays the same.

Q2.1 Defining languages (a)

$$\textsc{Long-Path} = \left\{ (G, s, t, k) : \begin{array}{l} G = (V, E) \text{ is a graph, } s, t \in V, \ k \text{ is a nonnegative integer,} \\ \text{and } G \text{ has a path from } s \text{ to } t \text{ with no repeated vertex} \\ \text{and at least } k \text{ edges} \end{array} \right\}.$$

A path with no cycle is a path that does not repeat a vertex. I added a number k to the input, so the question becomes yes/no: is there such a path from s to t with at least k edges? The original problem is a maximization problem, so I use "at least k."

Q2.2 Defining languages (b)

$$\textsc{Candy} = \left\{ (C, s, k) : \begin{array}{l} C = \{c_1, \ldots, c_n\} \text{ is a set of positive integers, } s \text{ and } k \text{ are nonnegative} \\ \text{integers, and there is a subset } I \subseteq C \text{ with } |I| \leq k \text{ and } \sum_{c \in I} c \geq s \end{array} \right\}.$$

In words, (C, s, k) is in the language if we can pick at most k candy pieces that have at least s grams of sugar in total. The original problem is a minimization problem, so I use "at most k."

Q3.1 ϒ's Scary P-NP Warmup (a)

V is not valid because it is not efficient: it does not run in polynomial time.

A polynomial-time verifier for L_{ACC} has to run in time polynomial in the size of the instance $(\langle M \rangle, x)$, and the certificate also has to have size polynomial in the size of the instance. V fails this for two reasons.

The running time is exponential in the size of c. The certificate c is an integer written in binary, so it takes only about $\log_2 c$ bits to write down, but V simulates M for c steps. A certificate with ℓ bits can be as large as $2^\ell - 1$, so V may take about 2^ℓ steps. This is exponential in the length of the input of V, not polynomial. So even if the certificate has polynomial length, the simulation can take exponential time in the size of the instance.

The certificate that is needed is not bounded by any polynomial. For $(\langle M \rangle, x) \in L_{\text{ACC}}$, the only certificates that make V accept are the integers $c \geq T$, where T is the number of steps M takes to accept x. T is not bounded by the size of $(\langle M \rangle, x)$: a small machine can run for a very large number of steps on a short input before it accepts. So there is no polynomial p such that every "yes" instance has an accepting certificate of length at most $p(|(\langle M \rangle, x)|)$.

V does satisfy the correctness condition, because M accepts x if and only if there is some c such that M accepts x within c steps. The problem is only the running time.

Q3.2 ϒ's Scary P-NP Warmup (b)

The certificate is the clique itself, given as a list of vertices.

Input: A graph $G = (V, E)$ and a certificate c
Output: accept if c is a clique of G with at least $|V|/2$ vertices, reject otherwise

```
1: function V_HC(⟨G⟩, c)
2:     if c is not a list of distinct vertices of V then return reject
3:     end if
4:     Let S be the set of vertices listed in c
5:     if |S| < |V|/2 then return reject
6:     end if
7:     for each pair of distinct vertices u, v ∈ S do
8:         if (u, v) ∉ E then return reject
9:         end if
10:    end for
11:    return accept
12: end function
```

Efficiency. Let $n = |V|$. A certificate that passes the first check lists at most n vertices, so its size is polynomial in the size of $\langle G \rangle$. Checking that the entries are distinct vertices of V takes $O(n^2)$ comparisons, comparing $|S|$ with $n/2$ takes $O(n)$ time, and the loop looks at $\binom{|S|}{2} = O(n^2)$ pairs, each needing one lookup in E, which takes at most $O(|E|)$ time. So V_{HC} runs in $O(n^2 |E|)$ time, which is polynomial in the size of $\langle G \rangle$.

Correctness. I show $\langle G \rangle \in$ HALF-CLIQUE $\iff \exists c$ such that $V_{\text{HC}}(\langle G \rangle, c)$ accepts.

($\Longrightarrow$) Suppose $\langle G \rangle \in$ HALF-CLIQUE. Then G has a clique $K \subseteq V$ with $|K| \geq |V|/2$. Let c be the list of the vertices of K, each written once. Then c is a list of distinct vertices of V, $|S| = |K| \geq |V|/2$, and every two distinct vertices of K are adjacent because K is a clique, so no check fails and $V_{\text{HC}}(\langle G \rangle, c)$ accepts.

($\Longleftarrow$) Suppose $V_{\text{HC}}(\langle G \rangle, c)$ accepts for some c. Then every check passed: c lists a set $S \subseteq V$, $|S| \geq |V|/2$, and every pair of distinct vertices of S is joined by an edge of E. So S is a clique of G of size at least $|V|/2$, and $\langle G \rangle \in$ HALF-CLIQUE.

So V_{HC} is an efficient verifier for HALF-CLIQUE, and HALF-CLIQUE $\in$ NP.

Q3.3 ϒ's Scary P-NP Warmup (c)

A set of $|V|-1$ vertices is V with one vertex removed. So there are only $|V|$ possible sets, and the decider can try all of them.

Input: A graph $G = (V, E)$
Output: accept if G has a clique of size $|V|-1$, reject otherwise

```
1: function D_BC(⟨G⟩)
2:     for each vertex w ∈ V do
3:         ok ← true
4:         for each pair of distinct vertices u, v ∈ V \ {w} do
5:             if (u, v) ∉ E then ok ← false
6:             end if
7:         end for
8:         if ok then return accept
9:         end if
10:    end for
11:    return reject
12: end function
```

Correctness. The subsets of V of size exactly $|V|-1$ are exactly the sets $V \setminus \{w\}$ for $w \in V$. In the iteration for w, the variable ok is true at the end of the inner loop if and only if every pair of distinct vertices of $V \setminus \{w\}$ is joined by an edge, i.e., if and only if $V \setminus \{w\}$ is a clique.

- If $\langle G \rangle \in$ BIG-CLIQUE, then G has a clique K with $|K| = |V| - 1$, so $K = V \setminus \{w\}$ for some $w \in V$. The outer loop either accepts before reaching w, or reaches w, finds ok true, and accepts. Either way D_{BC} accepts.
- If $\langle G \rangle \notin$ BIG-CLIQUE, then no set $V \setminus \{w\}$ is a clique, so in every iteration some pair is not adjacent, ok becomes false, and the iteration does not accept. After the outer loop ends, D_{BC} rejects.

Runtime. Let $n = |V|$. The outer loop runs at most n times. Each iteration looks at $\binom{n-1}{2} = O(n^2)$ pairs, and for each pair it checks whether $(u, v) \in E$, which takes at most $O(|E|) = O(n^2)$ time by scanning the edge list. So the total time is $O(n \cdot n^2 \cdot n^2) = O(n^5)$. The size of $\langle G \rangle$ is at least n, so this is polynomial in the input size. All loops are finite, so D_{BC} always halts. Therefore D_{BC} is a polynomial-time decider for BIG-CLIQUE, and BIG-CLIQUE $\in$ P.

Q4.1 Placing haunted houses (a)

The certificate is the set S of neighborhoods that get a haunted house.

Input: A graph $G = (V, E)$, an integer k with $0 \le k \le |V|$, and a certificate c
Output: accept if c is a set of at most k vertices to which every vertex has access, reject otherwise

1: **function** $V_{\text{HH}}((G, k), c)$
2: **if** c is not a list of distinct vertices of V **then return** reject
3: **end if**
4: Let S be the set of vertices listed in c
5: **if** $|S| > k$ **then return** reject
6: **end if**
7: **for** each vertex $v \in V$ **do**
8: **if** $v \notin S$ and there is no $u \in S$ with $(u, v) \in E$ **then return** reject
9: **end if**
10: **end for**
11: **return** accept
12: **end function**

Efficiency. Let $n = |V|$. A certificate that passes the first check lists at most n vertices, so its size is polynomial in the size of (G, k). Checking that c is a list of distinct vertices of V takes $O(n^2)$ comparisons, and comparing $|S|$ with k takes $O(n)$ time. The loop runs n times; each iteration checks membership of v in S in $O(n)$ time and, for each of the at most n vertices $u \in S$, looks up (u, v) in E in at most $O(|E|)$ time. The total is $O(n^2\,|E|)$, polynomial in the size of (G, k).

Correctness. I show $(G, k) \in$ Haunted-House $\iff \exists c$ such that $V_{\text{HH}}((G, k), c)$ accepts.

($\Longrightarrow$) Suppose $(G, k) \in$ Haunted-House. Then there is a set $S \subseteq V$ with $|S| \le k$ such that every $v \in V$ is in S or adjacent to some $u \in S$. Let c be the list of the vertices of S, each written once. Then the first two checks pass, and for every $v \in V$ the condition in the loop is false, so V_{HH} reaches the last line and accepts.

($\Longleftarrow$) Suppose $V_{\text{HH}}((G, k), c)$ accepts for some c. Then every check passed: c lists a set $S \subseteq V$, $|S| \le k$, and for every $v \in V$ either $v \in S$ or there is some $u \in S$ with $(u, v) \in E$. So every vertex has access to S, and $(G, k) \in$ Haunted-House.

Hence V_{HH} is an efficient verifier for Haunted-House, and Haunted-House $\in \mathsf{NP}$.

Q4.2 Placing haunted houses (b)

Take G to be a triangle, $V = \{a, b, c\}$ and $E = \{(a, b), (b, c), (a, c)\}$, with $k = 1$. This is a legal instance: $k = 1 \leq 3 = |V|$ and no vertex is isolated.

$(G, 1) \notin$ **Vertex-Cover.** A single vertex cannot cover all three edges: a does not cover (b, c), b does not cover (a, c), and c does not cover (a, b). The empty set covers nothing. So G has no vertex cover of size at most 1.

$f(G, 1) = (G, 1) \in$ **Haunted-House.** Take $S = \{a\}$. Then $a \in S$, and b and c are both adjacent to a, so every vertex has access to S, and $|S| = 1 \leq k$.

So $(G, 1) \notin$ Vertex-Cover but $f(G, 1) \in$ Haunted-House. This breaks the condition $x \in$ Vertex-Cover $\iff f(x) \in$ Haunted-House, so f is not a correct mapping reduction.

Q4.3 Placing haunted houses (c)

Let $n = |V|$ and $m = |E|$. The input (G, k) has size at least $n + m$ (the graph has to list its vertices and edges), plus $O(\log n)$ bits for $k \leq n$.

- Initializing G' with $V' = V$ and $E' = E$ copies the graph, which takes $O(n + m)$ time.
- The **for** loop runs once per edge of G, i.e., exactly m times. (It iterates over the original edge set E, not over E', so the new edges do not create more iterations.) Each iteration creates one new vertex and adds two new edges, which takes $O(1)$ time. So the loop takes $O(m)$ time.
- Returning (G', k) writes out the new graph and copies k. G' has $|V'| = n + m$ vertices and $|E'| = m + 2m = 3m$ edges, so the output has size $O(n + m)$ and writing it takes $O(n + m)$ time.

In total f runs in $O(n + m)$ time, which is polynomial in the size of the input (G, k).

Q4.4 Placing haunted houses (d)

Assume $(G, k) \in$ Vertex-Cover, and let $S \subseteq V$ be a vertex cover of G with $|S| \leq k$. I will show that $S' = S$ works for G'. Since $S \subseteq V \subseteq V'$, S' is a set of vertices of G', and $|S'| = |S| \leq k$. It remains to show that every vertex of V' has access to S'. There are two kinds of vertices in V'.

Original vertices $v \in V$. By assumption G has no isolated vertices, so v is an endpoint of some edge $(u, v) \in E$. Since S is a vertex cover, it contains an endpoint of this edge: $v \in S$ or $u \in S$. If $v \in S = S'$, then v has access to S'. If $u \in S = S'$, then v is adjacent to u in G', because $(u, v) \in E \subseteq E'$, so again v has access to S'.

New vertices w_{uv}, for $(u, v) \in E$. Since S is a vertex cover of G and $(u, v) \in E$, we have $u \in S$ or $v \in S$. The reduction added both edges (w_{uv}, u) and (w_{uv}, v) to E', so w_{uv} is adjacent in G' to a vertex of $S = S'$, and w_{uv} has access to S'.

So every vertex of V' has access to S' and $|S'| \leq k$, i.e., $f(G, k) = (G', k) \in$ Haunted-House.

Q4.5 Placing haunted houses (e)

Assume $(G', k) \in$ HAUNTED-HOUSE, and let $S' \subseteq V'$ be a set with $|S'| \leq k$ such that every vertex of V' has access to S'. S' may contain new vertices w_{uv}, which are not vertices of G, so I cannot use S' directly. Instead I replace each new vertex in S' by one endpoint of its edge.

For each edge $(u, v) \in E$, fix one of its endpoints and call it $e(w_{uv})$ (say, $e(w_{uv}) = u$). Define

$$S = (S' \cap V) \ \cup \ \left\{ e(w_{uv}) : w_{uv} \in S' \right\}.$$

$S \subseteq V$ and $|S| \leq k$. Every element of S is an original vertex. Each element of S' contributes at most one element to S: an original vertex contributes itself, and a new vertex w_{uv} contributes $e(w_{uv})$. So $|S| \leq |S'| \leq k$.

S is a vertex cover of G. Let $(u, v) \in E$ be any edge of G. The reduction created the vertex $w_{uv} \in V'$, and by assumption w_{uv} has access to S'. There are two cases.

- $w_{uv} \in S'$. Then $e(w_{uv}) \in S$, and $e(w_{uv})$ is an endpoint of (u, v), so S covers (u, v).
- $w_{uv} \notin S'$. Then w_{uv} is adjacent in G' to some vertex of S'. The only edges of G' that touch w_{uv} are (w_{uv}, u) and (w_{uv}, v), because the reduction adds only these two edges when it creates w_{uv}. So the only neighbors of w_{uv} are u and v, and hence $u \in S'$ or $v \in S'$. Both are original vertices, so $u \in S' \cap V \subseteq S$ or $v \in S' \cap V \subseteq S$, and S covers (u, v).

In both cases S contains an endpoint of (u, v). Since the edge was arbitrary, S is a vertex cover of G.

So G has a vertex cover of size at most k, i.e., $(G, k) \in$ VERTEX-COVER.

Q5.1 Understanding P, NP, and polynomial-time mapping reductions (a)

The missing pseudocode runs M on x and returns the same answer. The certificate is not used.

```
1: function V(x, c)
2:     Run M on x                                   ▷ c is ignored
3:     if M accepts x then return accept
4:     else return reject
5:     end if
6: end function
```

V is efficient. M is an efficient decider, so there is a polynomial p such that M halts on x within $p(|x|)$ steps. V runs M on x once and then does constant extra work, and it never reads c. So V runs in time $O(p(|x|))$, polynomial in $|x|$, whatever c is.

V is correct. I show $x \in L \iff \exists c$ such that $V(x, c)$ accepts.

($\Longrightarrow$) Suppose $x \in L$. Since M decides L, M accepts x. Then $V(x, c)$ accepts for every c, for example $c = \varepsilon$, which has size 0 (polynomial in $|x|$). So there exists a c such that $V(x, c)$ accepts.

($\Longleftarrow$) I prove the contrapositive. Suppose $x \notin L$. Since M decides L, M rejects x. Then $V(x, c)$ rejects for every c, because V's answer does not depend on c. So there is no c such that $V(x, c)$ accepts.

Hence V is an efficient verifier for L, and $L \in \mathsf{NP}$. Since $L \in \mathsf{P}$ was arbitrary, $\mathsf{P} \subseteq \mathsf{NP}$.

Q5.2 Understanding P, NP, and polynomial-time mapping reductions (b)

Since $L \in \mathsf{NP}$, there is an efficient verifier V for L: there is a polynomial p such that $V(x, c)$ halts within $p(|x|)$ steps for every x and c, and $x \in L$ if and only if $V(x, c)$ accepts for some c. The decider tries all certificates up to length $p(|x|)$.

Input: A string x
Output: accept if $x \in L$, reject otherwise

```
1: function D(x)
2:     for each string c of length at most p(|x|) do
3:         if V(x, c) accepts then return accept
4:         end if
5:     end for
6:     return reject
7: end function
```

D **always halts.** The alphabet is finite, so there are finitely many strings of length at most $p(|x|)$, and each call $V(x, c)$ halts because V is a verifier.

D **is correct.**

- If $x \in L$, then $V(x, c)$ accepts for some certificate c. We may take $|c| \leq p(|x|)$: V makes at most $p(|x|)$ steps, so it reads at most the first $p(|x|)$ symbols of c, and it behaves the same way on the prefix of c of that length. The loop reaches this certificate (unless it has accepted earlier), so D accepts.
- If $x \notin L$, then $V(x, c)$ rejects for every c, so the loop never accepts and D rejects.

So D decides L. Since $L \in \mathsf{NP}$ was arbitrary, every language in NP is decidable.

Q5.3 Understanding P, NP, and polynomial-time mapping reductions (c)

Assume $A \leq_p B$. By the first definition there is a polynomial-time computable function f such that $x \in A \iff f(x) \in B$ for every string x.

To show $A \leq_T B$, I must give a Turing machine that decides A using an oracle for B. Let O be an oracle that decides B: O halts on every input, and $O(y)$ accepts if and only if $y \in B$.

Input: A string x
Output: accept if $x \in A$, reject otherwise

```
1: function M_A(x)
2:     Compute y ← f(x)
3:     return O(y)
4: end function
```

M_A **always halts.** f is computable in polynomial time, so computing $f(x)$ halts (after at most polynomially many steps). Then M_A makes one call to O, which halts because O decides B, and returns its answer.

Correctness. By the property of f, the fact that O decides B, and the code of M_A,

$$\begin{aligned} x \in A &\implies f(x) \in B \\ &\implies O(f(x)) \text{ accepts} \\ &\implies M_A(x) \text{ accepts,} \end{aligned}$$

$$\begin{aligned} x \notin A &\implies f(x) \notin B \\ &\implies O(f(x)) \text{ rejects} \\ &\implies M_A(x) \text{ rejects.} \end{aligned}$$

So M_A decides A given an oracle for B, that is, $A \leq_T B$.

Q5.4 Understanding P, NP, and polynomial-time mapping reductions (d)

Assume $A \leq_p B$ and $B \leq_p C$. Then there are functions f and g and polynomials p and q such that

- f is computable in time at most $p(|x|)$ on input x, and $x \in A \iff f(x) \in B$ for all x;
- g is computable in time at most $q(|y|)$ on input y, and $y \in B \iff g(y) \in C$ for all y.

I assume p and q have nonnegative coefficients, so they are nondecreasing. (If not, replace each coefficient by its absolute value. This only makes the bounds larger.)

Define $h(x) = g(f(x))$: first compute $y = f(x)$, then compute and output $g(y)$.

Correctness. For every x, using the property of f and then the property of g with $y = f(x)$,

$$x \in A \iff f(x) \in B \iff g(f(x)) \in C \iff h(x) \in C.$$

Running time. Let $n = |x|$. Computing $y = f(x)$ takes at most $p(n)$ steps. The time to compute $g(y)$ is at most $q(|y|)$, which is polynomial in $|y|$, not in n, so we need to bound $|y|$ in terms of n. A Turing machine writes at most one tape cell per step, so after at most $p(n)$ steps the tape of the machine computing f holds at most $n + p(n)$ symbols (the n input cells plus at most $p(n)$ newly written ones). Hence

$$|y| = |f(x)| \leq n + p(n).$$

Since q is nondecreasing, computing $g(y)$ takes at most $q(|y|) \leq q(n + p(n))$ steps. The total time to compute $h(x)$ is at most

$$p(n) + q(n + p(n)).$$

Plugging a polynomial into a polynomial gives a polynomial: if p has degree a and q has degree b, then $q(n+p(n))$ has degree at most $\max(a, 1) \cdot b$. The sum of two polynomials is also a polynomial. So h is computable in time polynomial in $n = |x|$, the size of the input to A.

Therefore h is a polynomial-time computable function with $x \in A \iff h(x) \in C$, i.e., $A \leq_p C$.

Q6 Trick-or-Treating is (NP-)Hard

I show Ham-Cycle $\leq_p$ TSP. Since Ham-Cycle is NP-hard (lecture), this proves that TSP is NP-hard.

Step 1: the reduction. Given an unweighted graph $G = (V, E)$ with $n = |V|$ vertices, keep the same vertices, make the graph complete, give weight 1 to the pairs that are edges of G and weight 2 to the pairs that are not, and set the budget to n.

Input: An unweighted graph $G = (V, E)$
Output: A weighted complete graph G' and a budget k

```
function f(G)
    n ← |V|
    Initialize G' = (V, E') with E' = ∅
    for each pair of distinct vertices u, v ∈ V do
        Add the edge (u, v) to E'
        if (u, v) ∈ E then w(u, v) ← 1
        else w(u, v) ← 2
        end if
    end for
    return (G', k = n)
end function
```

G' is a complete weighted graph on V, so (G', n) is an instance of TSP.

Step 2: polynomial time. The size of G is at least n. The loop runs $\binom{n}{2} = O(n^2)$ times, and each iteration adds one edge and does one lookup in E, which takes at most $O(|E|) = O(n^2)$ time. Writing $k = n$ takes $O(\log n)$ time. So f runs in $O(n^4)$ time, polynomial in the size of G.

In both directions I use this fact: a cycle that visits each of the n vertices exactly once has exactly n edges.

Step 3a: $G \in$ Ham-Cycle $\implies (G', n) \in$ TSP. Assume $G \in$ Ham-Cycle, and let H be a cycle in G that visits every vertex exactly once. H uses n edges, all of them in E. G' has the same vertex set and contains every pair as an edge, so H is also a cycle in G' that visits every vertex exactly once. Each edge of H is in E, so it has weight 1 in G', and the total weight of H in G' is $n \cdot 1 = n \leq k$. So $(G', n) \in$ TSP.

Step 3b: $(G', n) \in$ TSP $\implies G \in$ Ham-Cycle. Assume $(G', n) \in$ TSP, and let T be a cycle in G' of total weight at most n that visits every vertex exactly once. T uses exactly n edges, and every edge of G' has weight 1 or 2. If T used an edge of weight 2, its total weight would be at least $(n-1) \cdot 1 + 2 = n + 1 > n$, a contradiction. So every edge of T has weight 1. By construction, an edge of G' has weight 1 only if it is an edge of G. So all n edges of T are in E, which means T is a cycle in G that visits every vertex exactly once. So $G \in$ Ham-Cycle.

By Steps 3a and 3b, $G \in$ Ham-Cycle $\iff f(G) = (G', n) \in$ TSP, and by Step 2, f is computable in polynomial time. So Ham-Cycle $\leq_p$ TSP, and since Ham-Cycle is NP-hard, TSP is NP-hard.

Q7.1 Subset Sum (a)

Step 3: It constructs a target sum s that has $m+n$ decimal digits, where the first m digits are all **3** and the last n digits are all **1**.

That is,

$$s = \underbrace{33\cdots3}_{m}\,\underbrace{11\cdots1}_{n}.$$

Q7.2 Subset Sum (b)

Here $m = 2$ and $n = 3$, so by part (a)

$$s = 33111.$$

The formula $\phi = (x_1 \vee \overline{x_2} \vee x_3) \wedge (\overline{x_1} \vee x_2 \vee \overline{x_3})$ is satisfied by $x_1 = x_2 = x_3 = \text{true}$: the first clause is true because of x_1 (and x_3), and the second because of x_2. So I take t_1, t_2, t_3, and then add a_1, a_2, b_2 to make the first two digits equal to 3:

$$I = \{t_1, t_2, t_3, a_1, a_2, b_2\} = \{10100,\ 1010,\ 10001,\ 10000,\ 1000,\ 1000\}.$$

Number	$j=1$	$j=2$	$i=1$	$i=2$	$i=3$
t_1	1	0	1	0	0
t_2	0	1	0	1	0
t_3	1	0	0	0	1
a_1	1	0	0	0	0
a_2	0	1	0	0	0
b_2	0	1	0	0	0
sum	3	3	1	1	1

Check:

$$10100 + 1010 + 10001 + 10000 + 1000 + 1000 = 33111 = s.$$

I is a submultiset of A: it uses one of the two copies of 10000 and both copies of 1000. So $(A, s) \in$ Subset-Sum.

Q7.3 Subset Sum (c)

$\phi' = (x_1 \vee x_1 \vee x_1) \wedge (\overline{x_1} \vee \overline{x_1} \vee \overline{x_1})$ has $n = 1$ variable and $m = 2$ clauses, so all numbers have $m + n = 3$ digits. The literal x_1 appears in c_1 only, and $\overline{x_1}$ appears in c_2 only.

Number	$j = 1$	$j = 2$	$i = 1$
a_1	1	0	0
b_1	1	0	0
a_2	0	1	0
b_2	0	1	0
t_1	1	0	1
f_1	0	1	1
s	3	3	1

The Subset-Sum instance is

$$(A = \{100,\ 100,\ 10,\ 10,\ 101,\ 11\},\ s = 331).$$

This instance is not in Subset-Sum. The sum of all six numbers in A is $100 + 100 + 10 + 10 + 101 + 11 = 332$. A submultiset I of A with sum 331 would have to leave out numbers that sum to exactly $332 - 331 = 1$. Every number in A is at least 10, so this is impossible. Hence no submultiset of A sums to s.

This is what we expect, because ϕ' is not satisfiable.

Q7.4 Subset Sum (d)

I number the digits $1, \ldots, m+n$ from the left, as in the table. I call the first m digits the *clause digits* and the last n digits the *variable digits*. I first prove a fact that I use in this part and in part (e).

No-carry fact. *For any submultiset $I \subseteq A$ and any position d, the dth digit of $\sum_{a \in I} a$ equals the number of elements of I whose dth digit is* 1.

Proof: every digit of every number in A is 0 or 1. So it is enough to show that at most 9 numbers of A have a 1 in the same position, because then adding a column never gives a carry. In a variable digit $m+i$, only t_i and f_i have a 1. In a clause digit j, the numbers with a 1 are a_j, b_j, and those t_i or f_i whose literal (x_i or $\overline{x_i}$, respectively) appears in c_j. Different such numbers correspond to different literals of c_j, and c_j has three literals, so there are at most $2+3=5$ numbers with a 1 in digit j. In both cases the count is at most $5 \leq 9$. □

Now assume $\phi \in$ 3SAT, and let α be a satisfying assignment of ϕ. I will use α to build a submultiset $I \subseteq A$ with sum s.

Variable numbers. For each $i = 1, \ldots, n$: if $\alpha(x_i) =$ true, put t_i into I; if $\alpha(x_i) =$ false, put f_i into I. Call the number chosen for x_i the *selected* number u_i. So u_i has a 1 in clause digit j exactly when the literal of x_i that is true under α appears in c_j.

Clause numbers. For each $j = 1, \ldots, m$, let r_j be the number of selected numbers $u_1, \ldots, u_n$ whose jth digit is 1.

- $r_j \geq 1$: α satisfies c_j, so c_j contains a literal that is true under α. If this literal is x_i, then $\alpha(x_i) =$ true, so $u_i = t_i$, and the jth digit of t_i is 1 because x_i appears in c_j. If it is $\overline{x_i}$, then $\alpha(x_i) =$ false, so $u_i = f_i$, and the jth digit of f_i is 1 because $\overline{x_i}$ appears in c_j.
- $r_j \leq 3$: selected numbers with a 1 in digit j correspond to different variables whose literals appear in c_j, and c_j has only three literals.

So $3 - r_j \in \{0, 1, 2\}$. If $3 - r_j = 1$, put a_j into I; if $3 - r_j = 2$, put both a_j and b_j into I; if $3 - r_j = 0$, put neither.

The sum of I is s. By the no-carry fact it is enough to count the 1s of I in each position.

- Variable digit $m+i$: the only numbers of A with a 1 here are t_i and f_i, and I contains exactly one of them. So this digit of the sum is 1.
- Clause digit j: the numbers of I with a 1 here are the r_j selected numbers counted above, plus the $3 - r_j$ numbers among a_j, b_j that were added. (The numbers $a_{j'}, b_{j'}$ for $j' \neq j$ have a 0 here.) So this digit of the sum is $r_j + (3 - r_j) = 3$.

So the sum of I has 3 in each of the first m digits and 1 in each of the last n digits, which is exactly s.

Therefore $I \subseteq A$ and $\sum_{a \in I} a = s$, i.e., $f(\phi) = (A, s) \in$ Subset-Sum.

Q7.5 Subset Sum (e)

Assume $(A, s) \in$ Subset-Sum, and let $I \subseteq A$ be a submultiset with $\sum_{a \in I} a = s$. I will use I to build a satisfying assignment of ϕ. (A, s) is the instance constructed from ϕ, so the no-carry fact from part (d) applies. By that fact, for each position d, the number of elements of I with a 1 in digit d equals the dth digit of s: it is 3 for each clause digit and 1 for each variable digit.

I contains exactly one of t_i, f_i for each i. The only numbers of A with a 1 in the variable digit $m + i$ are t_i and f_i. Exactly one element of I has a 1 in this digit, so exactly one of t_i, f_i is in I.

The assignment. Define α by

$$\alpha(x_i) = \begin{cases} \text{true} & \text{if } t_i \in I, \\ \text{false} & \text{if } f_i \in I. \end{cases}$$

By the previous paragraph exactly one of the two cases holds for each i, so α is a well-defined assignment to $x_1, \ldots, x_n$.

α satisfies every clause. Fix a clause c_j. Exactly three elements of I have a 1 in the clause digit j. Among the clause numbers, only a_j and b_j have a 1 in digit j, so they account for at most two of the three. Hence at least one of the three is a variable number: there is some i such that $t_i \in I$ or $f_i \in I$, and that number has a 1 in digit j.

- If $t_i \in I$ and the jth digit of t_i is 1: by construction the literal x_i appears in c_j, and $\alpha(x_i) = \text{true}$ because $t_i \in I$. So c_j contains a true literal.
- If $f_i \in I$ and the jth digit of f_i is 1: by construction the literal $\overline{x_i}$ appears in c_j, and $\alpha(x_i) = \text{false}$ because $f_i \in I$ (and so $t_i \notin I$). So $\overline{x_i}$ is true under α, and c_j contains a true literal.

In both cases α satisfies c_j. Since c_j was arbitrary, α satisfies every clause of ϕ.

Therefore ϕ is satisfiable, i.e., $\phi \in$ 3SAT.

Q8 (Optional extra credit)

Since $L_1, L_2 \in \mathsf{NP} \cap \mathsf{coNP}$, there are four efficient verifiers:

- V_1 for L_1 and $\overline{V_1}$ for $\overline{L_1}$;
- V_2 for L_2 and $\overline{V_2}$ for $\overline{L_2}$.

Write $L = L_1 \oplus L_2$. For every x,

$$x \in L \iff (x \in L_1 \text{ and } x \in \overline{L_2}) \text{ or } (x \in \overline{L_1} \text{ and } x \in L_2),$$
$$x \in \overline{L} \iff (x \in L_1 \text{ and } x \in L_2) \text{ or } (x \in \overline{L_1} \text{ and } x \in \overline{L_2}).$$

Each of the four cases can be checked by two of the verifiers. So my certificate says which case holds and gives one certificate for each of the two verifiers.

function $V(x, c)$ ▷ verifier for $L_1 \oplus L_2$
    if c is not of the form (b, c_1, c_2) with $b \in \{1, 2\}$ **then return** reject
    end if
    if $b = 1$ and $V_1(x, c_1)$ accepts and $\overline{V_2}(x, c_2)$ accepts **then return** accept
    end if
    if $b = 2$ and $\overline{V_1}(x, c_1)$ accepts and $V_2(x, c_2)$ accepts **then return** accept
    end if
    return reject
end function

function $\overline{V}(x, c)$ ▷ verifier for $\overline{L_1 \oplus L_2}$
    if c is not of the form (b, c_1, c_2) with $b \in \{1, 2\}$ **then return** reject
    end if
    if $b = 1$ and $V_1(x, c_1)$ accepts and $V_2(x, c_2)$ accepts **then return** accept
    end if
    if $b = 2$ and $\overline{V_1}(x, c_1)$ accepts and $\overline{V_2}(x, c_2)$ accepts **then return** accept
    end if
    return reject
end function

Efficiency. Each of V and $\overline{V}$ parses c and makes at most four calls to verifiers that run in time polynomial in $|x|$, so it runs in polynomial time. The certificates c_1, c_2 needed below have size polynomial in $|x|$, so $c = (b, c_1, c_2)$ does too.

V is correct, so $L \in \mathsf{NP}$.

($\Longrightarrow$) Suppose $x \in L$. If $x \in L_1$ and $x \notin L_2$, there are c_1, c_2 such that $V_1(x, c_1)$ and $\overline{V_2}(x, c_2)$ accept, so $V(x, (1, c_1, c_2))$ accepts. If $x \notin L_1$ and $x \in L_2$, there are c_1, c_2 such that $\overline{V_1}(x, c_1)$ and $V_2(x, c_2)$ accept, so $V(x, (2, c_1, c_2))$ accepts.

($\Longleftarrow$) Suppose $V(x, c)$ accepts for some $c = (b, c_1, c_2)$. If it accepted with $b = 1$, then $V_1(x, c_1)$ and $\overline{V_2}(x, c_2)$ accept, so $x \in L_1$ and $x \in \overline{L_2}$, because a verifier accepts some certificate only for strings in its language. If it accepted with $b = 2$, then in the same way $x \in \overline{L_1}$ and $x \in L_2$. In both cases x is in exactly one of L_1, L_2, so $x \in L$.

$\overline{V}$ is correct, so $\overline{L} \in \mathsf{NP}$, i.e., $L \in \mathsf{coNP}$.

($\Longrightarrow$) Suppose $x \in \overline{L}$, so x is in both of L_1, L_2 or in neither. If $x \in L_1$ and $x \in L_2$, there are c_1, c_2 such that $V_1(x, c_1)$ and $V_2(x, c_2)$ accept, so $\overline{V}(x, (1, c_1, c_2))$ accepts. If $x \notin L_1$ and $x \notin L_2$, there are c_1, c_2 such that $\overline{V_1}(x, c_1)$ and $\overline{V_2}(x, c_2)$ accept, so $\overline{V}(x, (2, c_1, c_2))$ accepts.

($\Longleftarrow$) Suppose $\overline{V}(x, c)$ accepts for some $c = (b, c_1, c_2)$. If $b = 1$, then $V_1(x, c_1)$ and $V_2(x, c_2)$ accept, so $x \in L_1 \cap L_2$. If $b = 2$, then $\overline{V_1}(x, c_1)$ and $\overline{V_2}(x, c_2)$ accept, so $x \notin L_1$ and $x \notin L_2$. In both cases x is not in exactly one of L_1, L_2, so $x \in \overline{L}$.

Hence $L_1 \oplus L_2 \in \mathsf{NP}$ and $L_1 \oplus L_2 \in \mathsf{coNP}$, that is, $L_1 \oplus L_2 \in \mathsf{NP} \cap \mathsf{coNP}$.